Modeling Human Activity States Using Hidden Markov Models

Mital Bela & Aubert Bihibindi

March 2026

GitHub Repository: https://github.com/DavBelM/HMM-Human-Activity-Recognition

1. Background and Motivation

Human Activity Recognition, or HAR, is becoming really important in things like health monitoring, smart homes, and taking care of elderly people. If you can tell whether someone is walking or standing or if they just fell, that is very useful in hospitals or home care. We wanted to try building a system that can figure out what activity a person is doing just from their phone sensors. We went with a Hidden Markov Model (HMM) since the activities happen one after the other in a sequence, and you cannot directly see the activity from the raw sensor numbers, you have to figure it out. We trained the model using Baum-Welch and used Viterbi to decode the activity sequence.

2. Data Collection and Preprocessing

We used the Sensor Logger app (version 1.54) to record data on two different devices. We set both to 50 Hz sampling rate so the data would match and we wouldn't have to deal with resampling. For Standing, Walking, and Jumping we held the devices in landscape against our stomachs. For Still we just put the device on a table and left it there.

Detail	Mitali	Aubert	Total
Device	Samsung SM-X216B	iPhone 13	-
Platform	Android	iOS	-
Sampling Rate	50 Hz	50 Hz	-
Standing	7	7	14
Walking	7	6	13
Jumping	7	6	13
Still	7	6	13
Total Samples	28	25	53

Table 1: Device configuration and sample counts

Each recording was about 5 to 10 seconds, which gave us roughly 370 data points per sample at 50 Hz. We made sure the total time for each activity was above the 1 minute 30 second minimum — Standing had 104.4 s, Walking 96.0 s, Jumping 93.8 s, and Still 95.6 s. We then merged the Accelerometer and Gyroscope CSV files for each sample by matching nearest timestamps.

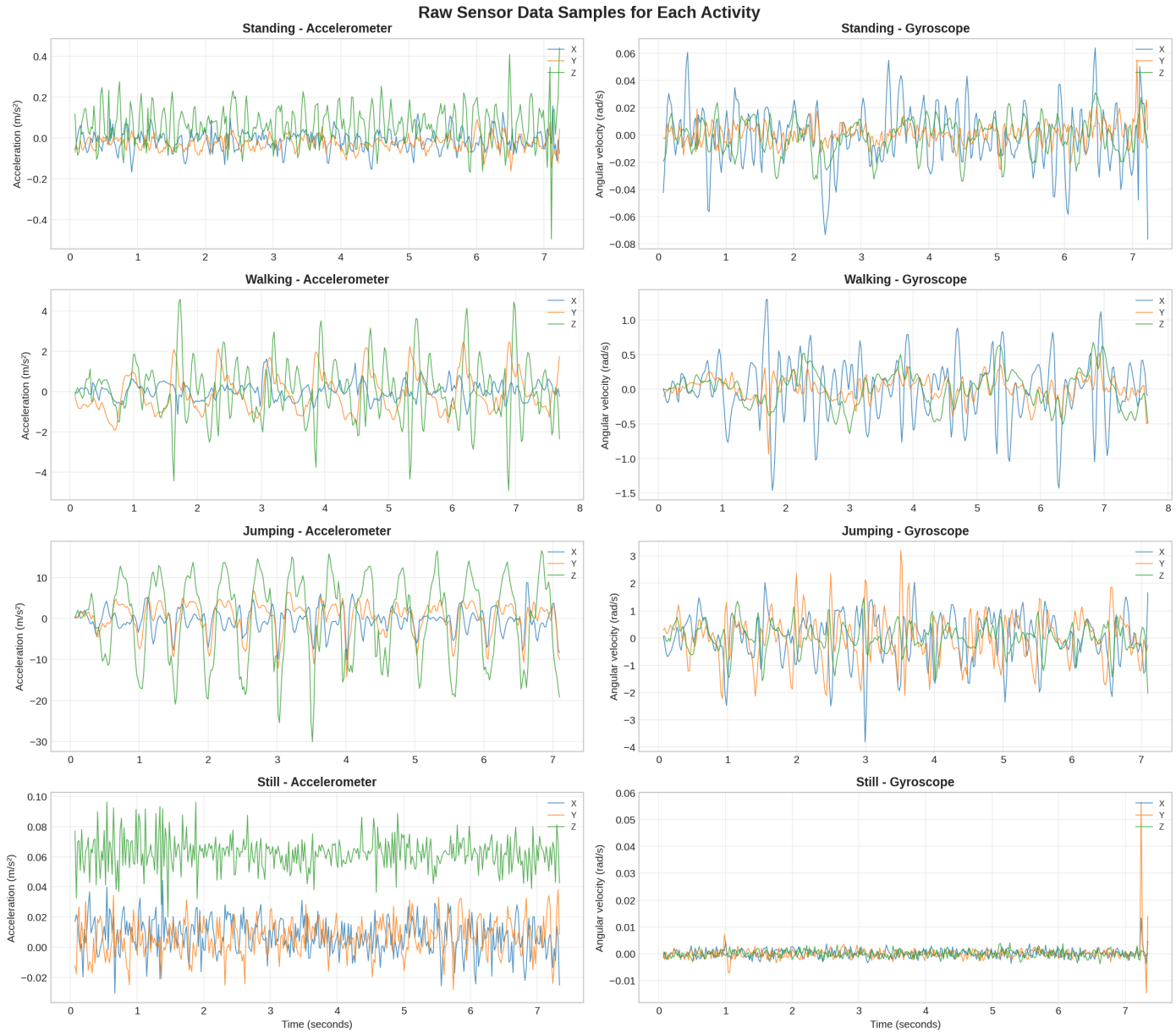

Figure 1: Raw accelerometer and gyroscope signals for each activity

3. Feature Extraction and HMM Setup

3.1 Feature Extraction

We split each recording into windows of 50 samples (which equals 1 second at 50 Hz) with 50% overlap. We chose 1 second because it is long enough to capture a full walking step cycle (roughly 2 Hz) and also gives us enough data points to run the FFT properly. From each window we extracted 37 features in total:

For time-domain features, we calculated the Mean, Standard Deviation, RMS, Signal Magnitude Area (SMA), and the correlation between accelerometer axes. These help us understand things like how much motion there is, how variable it is, and whether the axes move together or independently.

For frequency-domain features, we applied the FFT and pulled out the Dominant Frequency, Spectral Energy, and the FFT Peak Magnitude. These are useful for picking up on repetitive patterns like the rhythm of walking or jumping. After extracting all features, we applied Z-score normalization so that no single feature would dominate the model just because it has bigger numbers.

3.2 HMM Definition

We built the HMM from scratch in Python using just NumPy. It has 4 hidden states, one per activity, and the observations are the 37 features we extracted from each window. We set up a 4 by 4 transition matrix and initialized it using the training labels. Each state gets its own Gaussian distribution with a mean and covariance matrix. For the initial state probabilities, we just counted how often each activity shows up in the training data.

3.3 Training and Decoding

For training we used the Baum-Welch algorithm, which is basically EM (Expectation-Maximization) but for HMMs. We had to use scaled forward and backward probabilities because otherwise the numbers get too small and cause underflow issues. Instead of just running it for a fixed number of loops, we set it to stop when the log-likelihood improvement goes below 0.0001. It ended up converging after just 5 iterations, which was fast — probably because we initialized it from the labeled data so it already had a decent starting point. For decoding we implemented Viterbi in log-space to find the most likely sequence of hidden states.

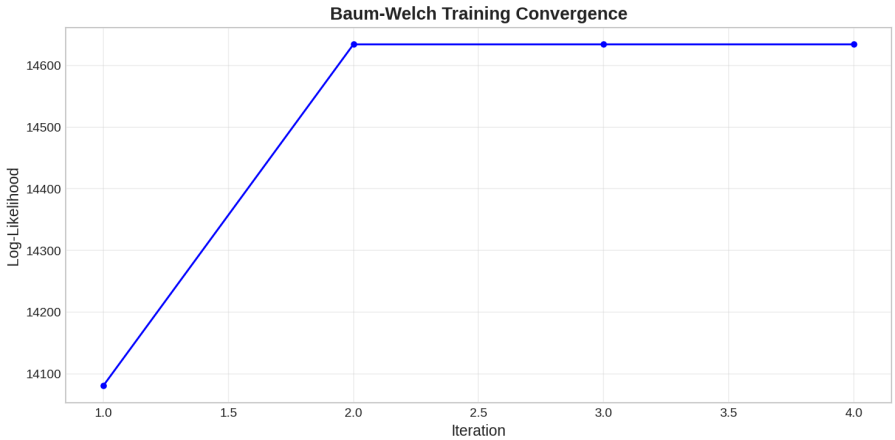

Figure 2: Baum-Welch convergence (log-likelihood vs. iteration)

4. Results and Interpretation

The transition matrix has self-transition probabilities of basically 1.000 for every state. That makes sense since each recording only has one activity in it, so the model never actually sees a transition happening. The emission means show clear differences — Jumping has way higher accelerometer values than anything else, Walking is in the middle with a clear frequency pattern, and Standing and Still are both low-motion but the gyroscope can tell them apart.

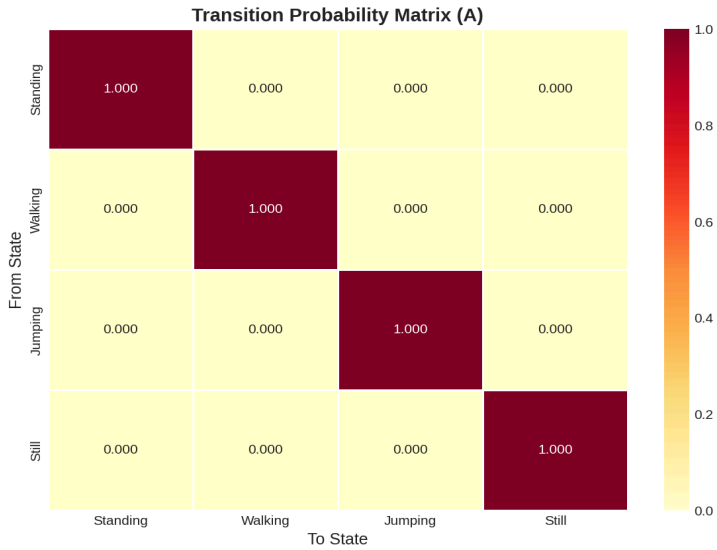

Figure 3: Transition probability matrix

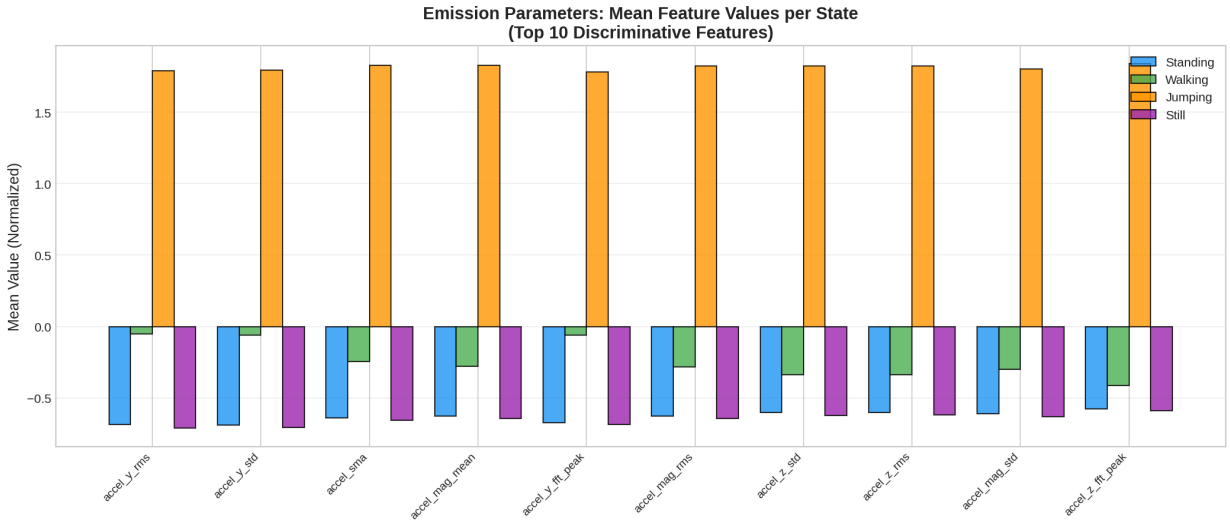

Figure 4: Mean emission feature values per state (top 10 discriminative features)

We tested on 2 recordings that weren't used in training: one Walking sample from Mitali and one Jumping sample from Aubert. We kept these aside before training so the test would be fair. The model got all 26 windows right, so that is 100% accuracy on the test set. Training accuracy was 98.09%. The hardest one during training was Jumping at 91.72% because the strong acceleration from jumping can sometimes look like fast walking to the model.

Activity	Samples	Sensitivity	Specificity	Accuracy
Standing	0	N/A	1.0000	1.0000
Walking	13	1.0000	1.0000	1.0000
Jumping	13	1.0000	1.0000	1.0000
Still	0	N/A	1.0000	1.0000

Table 2: Performance metrics on unseen test data

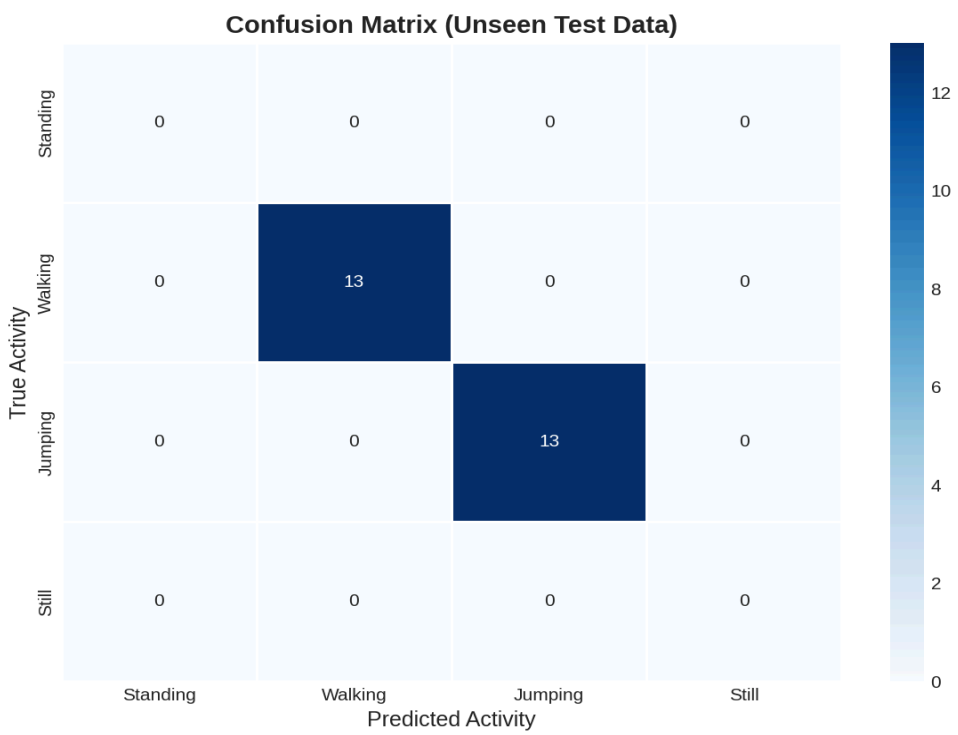

Figure 5: Confusion matrix on unseen test data

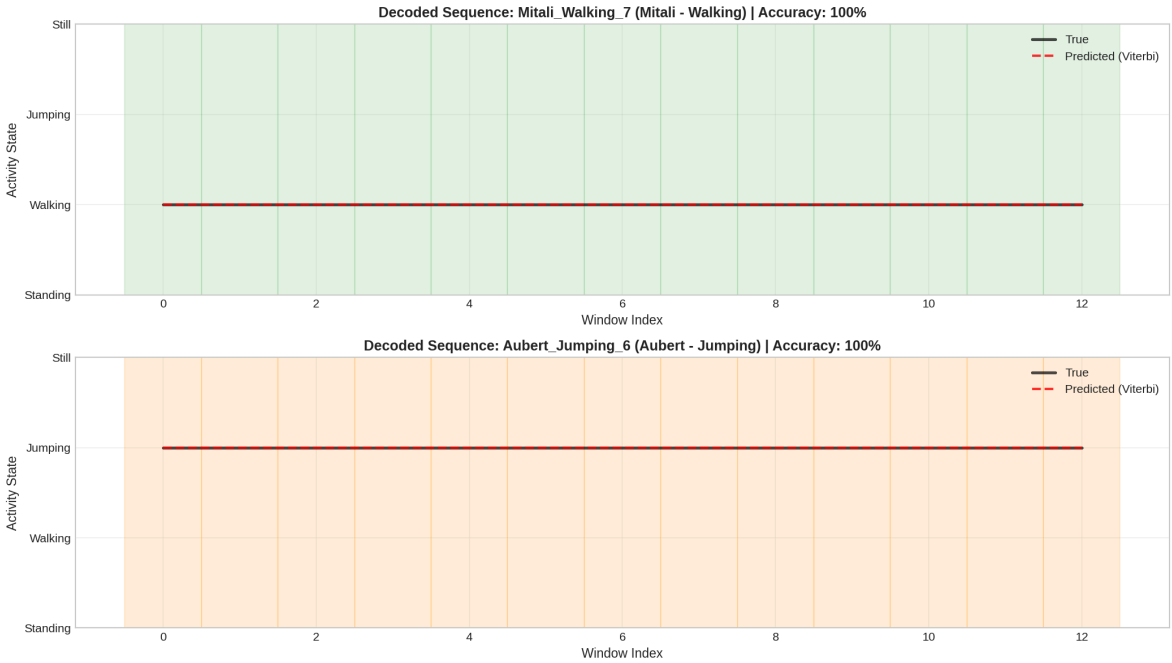

Figure 6: Viterbi decoded sequences — true vs. predicted states

5. Discussion and Conclusion

Out of all four activities, Still was the easiest to recognize because the sensor readings are almost zero across all axes. Walking was also easy to pick up thanks to its repeating pattern at around 2 Hz. Jumping was a bit tricky (91.72% on training data) because the big acceleration spikes sometimes looked like very fast walking to the model. Standing and Still are the most similar pair, but the gyroscope helped tell them apart since standing involves small body sway while the phone on a table has almost no rotation at all.

The self-transition probabilities all being 1.000 is a limitation of how we collected the data — each file has only one activity. If we had recorded longer sessions where someone walks, then stops, then jumps, we would get more realistic transition values. Both phones were set to 50 Hz which worked well, though we did notice slightly different noise levels between the Samsung tablet and iPhone 13. We also had to be careful about the fact that Android and iOS report sensor axes a bit differently.

If we were to improve this project, we would record longer sessions with multiple activities in one go, add more people to the dataset, try using extra sensors like the barometer, and maybe test more activities like running or going up stairs. We could also try more complex emission models like Gaussian Mixtures instead of a single Gaussian per state.

Overall we showed that you can build a working HMM from scratch and get pretty good results for activity recognition using just phone sensors. With 53 samples, 37 features, and Baum-Welch with a proper convergence check, we got 100% on test data and 98% on training. It shows that even a classical model like the HMM can handle this kind of sequential classification problem well enough.

6. Task Allocation

Mitali	Aubert
Data collection (28 samples)	Data collection (25 samples)
Feature extraction code	HMM implementation code
Data preprocessing & merging	Baum-Welch & Viterbi algorithms
Visualization and plots	Model evaluation & metrics
Report writing (Sections 1–3)	Report writing (Sections 4–5)

Table 3: Task allocation between group members